3. Advanced Styling

Contents

A show rule 1
New settings 2
`#document(...)` 2
`#page(...)` 2
`#text(...)` 2
`#align(...)` 2
(fill: ...) 2
New functions 3
`#grid(...)` 3
`#block(...)[...]` 3
`#place(...)[...]` 3
`#link(<string>)` 3
`#context` 4
`<...>.body` 4
Small commands 4
Delimiters 4
Questions: 5

A show rule

`#show <object>: <function>` applies the function to all the objects. To specify a subcategory of the object, you can use the method `<object>.where(<property>: <value>)`. Examples:

- `#show title: smallcaps` – makes title be in small caps.
- `#show heading: set align(center)` – makes headings centered.
- `#show heading.where(level: 2): set text(…)` – sets font of 2nd-level headings.

In `#show ...: <var> =>...` you can use the same `<var>` for different `#show`; it's local.

New settings

`#document(...)`

- `title: [text]` or `title: "text"` – gives its value to the `#title()`.
- `author: "text"`

`#page(...)`

- `paper: "us-letter"` – so-called “US Letter” paper size, analog of A4.
- `header: [<content>]` – sets a header (a note at the top of the page):
  - `header: align(…)[<content>]` – aligns the header.
- `numbering: <string>` – controls page numbers:
  - `numbering: "1"` – standard numbering: 1, 2, 3…
  - `numbering: "a"` – a, b, c…
  - `numbering: "(1/1)"` – (`<current page>/<how many pages>`).
- `columns: <number>` – divides text into columns.

`#text(...)`

- `weight: <string>`
  - `"bold"`
  - `"regular"`
- `style: <string>`
  - `"italic"`

`#align(...)`

- `#align(horizon)` – vertical centering.

(fill: …)

- `fill: <color>` – colors the object:
  - `#text(fill: red)` – red text.
  - `#rect(fill: black)` – black rectangle.

New functions

`#grid(...)`

Grid is a basic table without lines, e.g.: A B / C D

The arguments of the function:

- `columns: <...>` – sets columns:
  - `columns: <number>` – several columns with auto-width.
  - `columns: (<length_1>, <length_2>, ...)` – the first column with `<length_1>`, the second with `<length_2>`, etc.

Use a relative length `fr` to divide all available width proportionally.

- `align: <...>` – aligns columns:
  - `align: center` – all columns are centered.
  - `align: (left, center, right, ...)` – 1st column left-aligned, 2nd centered, 3rd right, etc.
- [`<content>`] – creates a cell.
  - `align(…)[<content>]` – aligning of an individual cell.

`#block(...)[...]`

Most objects, except paragraphs, are placed into `#block`. It allows us to control gaps before and after an object: `#show <object>: set block(above: <length>, below: <length>)`.

`#place(...)[...]`

The function places content at the top or bottom of a page regardless of other content. Its arguments are:

- `<alignment>` – e.g., `#place(top + center, ...)`
- `float: true` – switches on the collision with other content.
- `scope: "parent"` – puts the content outside current columns (across the whole page).
- `clearance: <length>` – a gap before or after.

`#link(<string>)`

- `#link("https://www.example.com/")` ⟶ https://www.example.com/.

- `#link("https://www.example.com/")[*Clickable text*]` ⟶ **Clickable text**.
- `#link("www.example.com/")` ↛ the link is broken, a protocol required.
- `#link("mailto:example@gmail.com")` ⟶ a link to an email.

`#context`

`#context <element>.<property>` – outputs the property's value, e.g.:

- `#context text.size` ⟶ `11pt`.
- `#context document.title` ⟶ 3. Advanced Styling.

`<...>.body`

Returns plain text of an element, without counters, boxing, or formatting:

- `#show heading: it => it.body` ⟶ the headings become plain text.
- `#context document.title.body` ⟶ Advanced Styling.

Small commands

- `#h(<length>)` – horizontal gap.
- `#v(<length>)` – vertical gap.
- `\` – new line.
- `// <comment>`
- `/* <comment> */`

Delimiters

Hyphenation is enabled when `#par(justify: true)`, otherwise it's off.

Quick way to put a text into **"..."**, **(...)**, **[...]**, or **{...}**, is to type it plain, then select it and type the delimiter.

- (...) – hold arguments of a function.
  - code mode ⟶ no # and markups allowed.
  - `#<function>()` means the function receives nothing.
  - can hold content in 2 (often equivalent) forms: `"<text>"` and `[<content>]`.

- [...] – hold content,
  - markup mode ⟶ # is required.
  - `#<function>[]` means the function receives something (empty content block).
  - often isn't necessary (syntactic sugar), because the content can go to (...), e.g.: `#place(top, rect()) = #place(top)[#rect()]`.
  - determine a scope, hence **empty scope** command **`#[…]`**.
- {...} – hold code block.
  - code mode ⟶ no # and markups allowed.
  - allow to span code across multiple lines.
  - different lines of a code block concatenate with each other:

```
it.body + [.]  =  {it.body
                  [.]}
```

Questions:

1. Explain `#show <object>: <function>` rule. Explain `#show <object>: set <…>` rule.
2. Explain `.where(…)` method.
3. How to make a text smallcap?
4. Can we use the same `<var>` in different `#show <…>: <var> =>...` rules?
5. Explain `title:` and `author:` arguments. Which values do they take? Which function do they belong to? How do they interact with `#title` command?
6. What's US Letter? How to make our pages have it?
7. What's the difference between header and heading?
8. How to make pages of a document to have headers? How to align a header?
9. Why are these arguments of `#page` not correct: `numbering: 1`, `numbering: a`, `numbering: (1/1)`. Which effect do their correct versions have?
10. How to divide pages into columns using `#set`?
11. Explain `weight:` and `style:` arguments. Which values do they take? Which function do they belong to?

12. Explain the difference between `center` and `horizon`.
13. Explain the `fill:` argument. Which values does it take? Which functions does it belong to?
14. Explain `#rect` function.
15. Explain `#grid` function. What does it produce? Which arguments does it take?
16. How to make a grid of several columns without any specified widths? How to divide all available space between several columns in some proportion? How to make a grid of several columns, each with specific width?
17. How to align all the grid the same way at once? How to align a grid by the columns? How to align an individual cell of a grid?
18. What's `fr`? Why do we call it a relative length?
19. Which function do `[ ]` play in the `#grid`?
20. What's the `#block`? What does `#show <...>: set block(...)` allows us to control and how?
21. Explain `#place` function. By explain I always mean syntax and functionality.
22. Explain `float:`, `scope:` and `clearance:` arguments. Which values do they take? Which function do they belong to?
23. Explain `#link` function. How to display a clickable link to a site in the document? How to make a clickable text linked to a site? How to make a link to an email?
24. Find errors:
    - `#link(https://www.example.com/)`
    - `#link("www.example.com/")`
    - `#link("mailto: example@gmail.com")`
25. Explain `#context` function and `<...>.body` method
26. Explain the commands:
    - `#[...]`
    - `#h`
    - `#v`
    - `\`
    - `//`

- `/*...*/`

27. What's the link between hyphenation and justification?
28. If you already typed some text, what's the easy/alternative way to put it inside delimiters such as `[], "", {}, ()`?
29. Explain main functionality of each `(), []` and `{}`. Which of them are code mode? Which of them allows `#` inside?
30. What's the difference between `#<function>()` and `#<function>[]`?
31. Why is `[]` often just a syntactic sugar?
32. Does `[]` determine its own scope? What does it even mean?
33. Which of the two forms is correct: `("text")` or `([<text>])`?
34. New line ends code execution. But what if I want to span code across several lines?
35. How to concatenate two strings if they are: a) on the same line; b) on different lines?